

Week 3 — Boolean Logic and Physical Gates

Target time: approximately 3 hours

Suggested rhythm: 70 minutes at the whiteboard · 65 minutes in the Logic Gate Lab · 30 minutes observing code · 15 minutes explaining from memory

A switch can be open or closed. That gives us a state.

But how could a pile of switches decide, “turn on only when both conditions are true”?

Why this week exists

We already know how a physical system can carry a bit:

```
continuous physical quantity
    ↓ engineered thresholds
stable LOW or HIGH
    ↓ agreed encoding
logical 0 or 1
```

That solves **representation**. It does not yet solve **computation**.

A loose switch is only a controllable break in a path. It does not inherently mean AND, OR, permission, equality, or addition. To compute, switches need:

1. an arrangement—series, parallel, inversion, or a larger network;
2. a convention—what voltage ranges count as 0 and 1;
3. a rule connecting input states to an output state;
4. restoration—each stage must produce a signal clean enough to drive the next stage.

The arrangement is physical. The rule is logical. A **logic gate** is where those two descriptions meet.

By the end of Week 3, you should be able to look at a Boolean function, write its truth table, imagine a physical switching network that realizes it, and explain why the circuit computes without understanding what its bits mean.

The larger path is:

```
switching device
  → gate
  → Boolean expression
  → combinational circuit
  → arithmetic and selection
  → ALU
  → CPU datapath
```

This week earns the first four arrows. We will glimpse one-bit addition, but save full adders, storage, clocks, and registers for their own treatment.

Part I — The historical problem: rules need machinery

Boole separated reasoning from the things being reasoned about

Human reasoning contains patterns such as:

```
the machine may run IF power is present AND the guard is closed
the alarm sounds IF smoke is detected OR heat is excessive
the lamp is on IF the switch is NOT open
```

George Boole showed that such statements could be treated algebraically. Instead of arguing about machines, smoke, or guards, we can use variables and operations:

```
P AND G
S OR H
NOT O
```

This was a major abstraction step. A variable could stand for any two-way proposition. The same algebra worked regardless of the proposition's meaning.

But an algebra on paper is not yet a computer. The next engineering question was:

```
Can a physical network obey the same algebra?
```

Shannon saw algebra inside relay circuits

Telephone networks used electromagnetic relays. A small current in a coil moved a contact, opening or closing another electrical path. Engineers already combined contacts to route calls and control equipment, but large networks were difficult to reason about systematically.

Claude Shannon made the crucial connection: a relay contact has two useful states, and networks of contacts can be described with Boolean algebra.

```
contact closed ↔ condition true ↔ 1
contact open   ↔ condition false ↔ 0
```

Two contacts in series require both to close: AND. Two contacts in parallel allow either path to conduct: OR. A normally closed contact opens when activated: NOT-like behavior.

The practical gain was not merely philosophical. A messy wiring problem could be written as an expression, simplified algebraically, and rebuilt with fewer contacts. Logic became a design language for physical machinery.

Why relays did not become the final switching element

Relays make the idea visible, but their contacts physically move. Movement takes time; contacts bounce, wear, make noise, and consume substantial space. Vacuum tubes removed mechanical motion but were bulky, hot, and power-hungry.

The transistor supplied a compact solid-state switch. In modern CMOS logic, MOSFETs are arranged so an input voltage controls conducting paths toward the power rail or ground. No single transistor is “the Boolean algebra.” The **network topology** makes the Boolean function.

That causal chain matters:

```
Boolean algebra: describe two-valued rules
  ↓
relay networks: embody those rules in controlled contacts
  ↓
transistors: embody them without moving contacts
  ↓
integrated circuits: compose enormous numbers of gates
```

Boole did not design a CPU, Shannon did not invent every switching circuit, and a transistor alone is not a computer. The breakthrough is the composition of ideas across layers.

Part II — From a switch to a function

A switch by itself does almost nothing

Imagine a battery, lamp, and one switch:

```
power —[ A ]— lamp — return
```

If **A** closes, current can flow. If **A** opens, it cannot. We can choose:

```
A open   → A = 0
A closed → A = 1
lamp off → Y = 0
lamp on  → Y = 1
```

Then this circuit follows $Y = A$. It copies the logical condition.

Now ask for a different rule: “lamp on only when A and B are both active.” A pile containing two switches does not answer that question. Their **arrangement** does.

Series contacts create an AND intuition

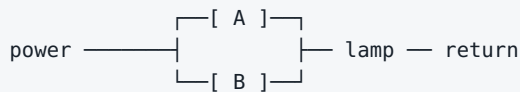
```
power —[ A ]—[ B ]— lamp — return
```

There is one path. A break at either contact stops current:

A	B	Y = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Read the table row by row. A **truth table** is a complete behavioral specification: it lists the output for every possible input combination. With two inputs there are $2^2 = 4$ rows.

Parallel contacts create an OR intuition



Either branch can complete a path:

A	B	Y = A OR B
0	0	0
0	1	1
1	0	1
1	1	1

This physical picture is useful, but do not stretch it too far. A real electronic gate is designed to produce valid output voltage levels, supply limited current to later gates, meet timing constraints, and tolerate noise. A lamp circuit is an intuition, not a transistor-level schematic.

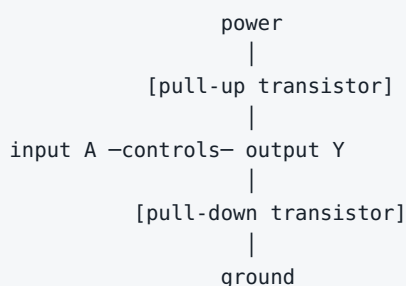
Inversion needs a complementary path

For NOT, we want the output active precisely when the input is inactive:

A	Y = NOT A
0	1
1	0

A relay can use a normally closed contact: when its coil is inactive, the contact conducts; energizing the coil opens it.

In a simplified CMOS inverter:



When **A** is LOW, the pull-up path conducts and drives **Y** HIGH. When **A** is HIGH, the pull-down path conducts and drives **Y** LOW. The devices are chosen and connected so that, in normal steady operation, one path wins and the other is off.

This is more than “blocking electricity.” The network actively drives the output toward a valid rail. That restoring behavior lets one imperfect physical signal become another clean logical signal.

Pause and predict

Without looking ahead:

1. What arrangement of ordinary contacts naturally means AND?
2. What arrangement naturally means OR?
3. Why can NOT not be explained as merely “put another ordinary switch in series”?
4. Which parts of these descriptions are physical, and which are conventions?

Part III — Six gates, derived rather than memorized

We will use these symbols:

NOT A	$\neg A$	or !A in many programming languages
A AND B	$A \wedge B$	or A & B for one-bit bitwise logic
A OR B	$A \vee B$	or A B for one-bit bitwise logic
A XOR B	$A \oplus B$	or A ^ B

Symbols vary by context. The truth table is the unambiguous definition.

NOT — opposite state

$$Y = \neg A$$

A	Y
0	1
1	0

Physical intuition: activating the control disconnects one route and enables its complement.
Logical use: invert a condition, form an active-low control, or turn “permission” into “deny.”

AND — every required condition

$$Y = A \wedge B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Physical intuition: series contacts. Logical use: enabled AND ready .

OR — at least one sufficient condition

$$Y = A \vee B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Physical intuition: parallel paths. Logical use: `interrupt_A OR interrupt_B`. OR is inclusive: both inputs being 1 still gives 1.

XOR — the inputs differ

$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XOR means **exactly one of two inputs** is 1, equivalently “the two bits are different.” That second phrasing scales better:

$$A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$$

Physical intuition: two mutually exclusive conducting cases—A without B, or B without A—joined by OR. XOR is more complex than plain series or parallel contacts because the circuit must reject both-equal cases.

Logical uses include toggling controlled bits, parity, inequality detection, and the low result bit of one-bit addition.

NAND — AND followed by NOT

$$Y = \neg(A \wedge B)$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Physical intuition: the output stays active unless both controlled conditions create the one state that pulls it inactive. In CMOS, NAND has a particularly natural transistor arrangement: the pull-down path uses two nMOS devices in series, so it can pull LOW only when both inputs are HIGH; a complementary parallel pull-up network handles the other rows.

NOR — OR followed by NOT

$$Y = \neg(A \vee B)$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Physical intuition: the output is active only while neither input creates an active path. In CMOS, the nMOS pull-down devices can be parallel, so either HIGH input pulls the output LOW; the complementary pull-up requires both inputs LOW.

One table to compare them

A	B	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

Do not memorize this as 24 disconnected cells. Derive each column from one sentence:

- AND: all inputs are 1.
- OR: at least one input is 1.
- XOR: the inputs differ.
- NAND: not all inputs are 1.
- NOR: no input is 1.

Part IV — Boolean expressions are circuit descriptions

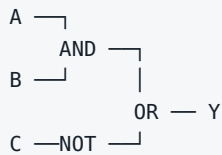
Consider:

$$Y = (A \wedge B) \vee \neg C$$

Read it as a recipe:

1. send **A** and **B** into an AND gate;
2. send **C** into a NOT gate;

3. OR those two intermediate results.



The wires carry intermediate Boolean values. Parentheses state grouping, just as they do in arithmetic.

Derive a truth table systematically

Name intermediate results:

$P = A \wedge B$
 $Q = \neg C$
 $Y = P \vee Q$

A	B	C	$P = A \wedge B$	$Q = \neg C$	$Y = P \vee Q$
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Three inputs produce $2^3 = 8$ rows. Intermediate columns turn a complicated guess into a sequence of small mechanical steps.

Expression, circuit, and truth table are three views

Boolean expression	compact symbolic rule
gate diagram	structural implementation
truth table	complete observable behavior

Different gate diagrams can implement the same truth table. This is important: hardware design is not merely finding a circuit; it is often finding one that meets constraints such as fewer transistors, shorter delay, lower power, or easier physical layout.

Part V — De Morgan's laws: moving inversion through a network

Two identities appear everywhere in hardware and software:

$$\neg(A \wedge B) = \neg A \vee \neg B$$

$$\neg(A \vee B) = \neg A \wedge \neg B$$

Say them in plain language:

- “not both” means “at least one is not.”
- “neither” means “the first is not and the second is not.”

Do not trust the slogan—verify all cases

A	B	$\neg(A \wedge B)$	$\neg A \vee \neg B$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

The output columns match in every row, so the expressions specify the same Boolean function.

For the second law:

A	B	$\neg(A \vee B)$	$\neg A \wedge \neg B$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Why hardware designers care

Suppose the available physical gate is NAND. De Morgan's laws let us reshape an expression around that primitive. They also help us reason about active-low signals, where an asserted condition is represented by LOW rather than HIGH.

The laws are not tricks that alter behavior. They prove that two different structures have the same input/output function.

Part VI — Universal gates: one kind can build every Boolean function

A gate set is **functionally complete** if it can express every Boolean function.

NAND alone is functionally complete.

Build NOT from NAND

Tie the two inputs together:

$$\text{NAND}(A, A) = \neg(A \wedge A) = \neg A$$

Build AND from NAND

Invert the NAND output using another NAND:

$$\begin{aligned} P &= \text{NAND}(A, B) = \neg(A \wedge B) \\ \text{AND}(A, B) &= \text{NAND}(P, P) \end{aligned}$$

Build OR from NAND

Use De Morgan:

$$\begin{aligned} A \vee B &= \neg(\neg A \wedge \neg B) \\ &= \text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B)) \end{aligned}$$

Once we can build NOT, AND, and OR, we can build any truth table by combining the input cases that should produce 1.

NOR is also universal:

$$\begin{aligned} \text{NOT } A &= \text{NOR}(A, A) \\ \text{OR} &= \text{invert a NOR result} \\ \text{AND} &= \text{NOR}(\text{NOR}(A, A), \text{NOR}(B, B)) \end{aligned}$$

“Universal” does not mean every chip is physically made from literal two-input NAND symbols. Real libraries contain many gate shapes optimized for electrical and layout constraints. Universality is a statement about what can be expressed.

Part VII — Combinational logic: outputs depend on inputs now

A **combinational circuit** has no intentional memory. Its output is a Boolean function of its current inputs:

$$Y = f(A, B, C, \dots)$$

Change an input and physical effects propagate through the gates. After the finite propagation delay, the outputs settle to the new function value.

Examples:

- a decoder selects one output from an encoded input;
- a multiplexer chooses one data input;
- a comparator reports equality or ordering;
- an adder computes result and carry bits;
- an ALU combines arithmetic and logic functions.

The word “now” is an abstraction. Real transistors do not respond instantaneously. During transitions, intermediate outputs may briefly differ from the final truth-table result. Later we will care about propagation delay, hazards, clocks, and state. For Boolean reasoning this week, we analyze settled values.

What combinational logic cannot do

If every input returns to its old value, a pure combinational circuit returns to its old output. It cannot remember that something happened earlier.

Memory requires another idea—feedback and controlled sampling. Do not smuggle memory into this week's model.

Part VIII — A glimpse of arithmetic: the half-adder

Add two one-bit numbers:

A	B	binary total	Sum	Carry
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	10	0	1

Compare the output columns with the gate tables:

Sum = A XOR B
Carry = A AND B

A ——— XOR — Sum
|
B ——— AND — Carry

This is a **half-adder**. It adds two bits, but it does not accept a carry from a less-significant position. That limitation is exactly why it is only a preview.

Later we will ask:

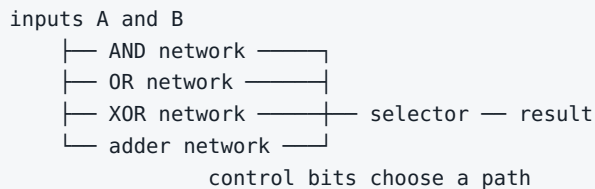
- How do we include carry-in?
- How do many one-bit stages form a wider adder?
- How long does carry take to propagate?
- How does a circuit remember operands or results?

Those questions lead to full adders, registers, timing, and storage. Stop here for now and preserve that conceptual space.

Part IX — From gates toward an ALU and CPU

An arithmetic logic unit does not “know arithmetic” in a human sense. It contains circuits whose physical topology implements useful Boolean functions.

One route toward a simplified ALU is:



Replicate one-bit slices across a word, add carry connections and status logic, and the machinery can operate on multi-bit values. Put registers around the datapath, add a clock and control logic, and we approach a CPU execution unit.

But preserve the layers:

transistors do not see integers
gates do not see C variables
the ALU does not understand "salary" or "pixel"
the CPU does not understand the purpose of your program

Each layer follows local physical and encoding rules. Meaning is supplied by the system built above it.

Part X — Hands-on Logic Gate Lab

Open this local, offline challenge in a browser:

`weekly/challenges/week-003-logic-gate-lab.html`

Use this loop:

1. **Predict** an output before touching the controls.
2. **Set** the input switches.
3. **Observe** the gate and truth-table row.
4. **Explain** the result in one sentence.
5. **Build** the requested expression and complete the scored challenge.

The lab deliberately hides challenge answers until you submit an attempt. Explore mode may show gate behavior, but do not use it to bypass your prediction.

Suggested time:

- 20 minutes: gate explorer and physical switch intuition;
- 20 minutes: expression builder and De Morgan comparison;
- 25 minutes: scored prediction rounds.

Part XI — Observe the same functions in Python

This optional exercise makes a useful distinction: Python simulates the truth table in software; a physical gate realizes it through electrical behavior.

Predict the rows before running:

```
python3 - <<'PY'
def bit_not(a):
    return 1 - a

def gates(a, b):
    return {
        "NOT A": bit_not(a),
        "AND": a & b,
        "OR": a | b,
        "XOR": a ^ b,
        "NAND": bit_not(a & b),
        "NOR": bit_not(a | b),
    }

for a in (0, 1):
    for b in (0, 1):
        print(a, b, gates(a, b))
PY
```

Now verify De Morgan's laws exhaustively:

```
python3 - <<'PY'
for a in (0, 1):
    for b in (0, 1):
        law1 = (not (a and b)) == ((not a) or (not b))
        law2 = (not (a or b)) == ((not a) and (not b))
        print(f"A={a} B={b} law1={int(law1)} law2={int(law2)}")
PY
```

Every line should report both laws as true. The program did not prove the laws for arbitrary multi-valued systems; it enumerated every state in this two-valued domain.

Optional small C observation

```
#include <stdio.h>

int main(void)
{
    puts("A B | AND OR XOR NAND NOR");
    for (int a = 0; a <= 1; ++a) {
        for (int b = 0; b <= 1; ++b) {
            printf("%d %d | %d %d %d %d %d\n",
                a, b, a && b, a || b, a != b,
                !(a && b), !(a || b));
        }
    }
    return 0;
}
```

Build with warnings enabled:

```
gcc -std=c17 -Wall -Wextra -O0 -g logic_table.c -o logic_table
./logic_table
```

In C, `&&`, `||`, and `!` are logical operators with short-circuit behavior; `&`, `|`, `^`, and `~` operate bitwise on integer representations. For inputs restricted to 0 and 1, several result columns coincide, but the language semantics are not interchangeable in general.

Part XII — Mastery checkpoint

Close the lesson and answer:

How can a physical circuit implement a Boolean function without understanding it?

A strong answer should connect all of these:

1. Input meanings are encoded as physical voltage ranges.
2. Transistors or relay contacts respond locally to physical conditions.
3. Their arrangement constrains which conducting paths exist for each input combination.
4. Those paths drive an output into a physical state interpreted as 0 or 1.
5. The resulting input/output mapping matches a truth table.
6. No component needs a concept of the application-level meaning; the “function” is our abstract description of deterministic physical behavior.

Do not memorize that list. Rebuild the explanation from a concrete example, such as series contacts implementing AND.

Explain

Without notes:

- derive NOT, AND, OR, XOR, NAND, and NOR;
- explain why OR is inclusive and XOR is different;
- state both De Morgan laws in plain language;
- explain why NAND or NOR can be universal.

Draw

On one page:

```
two physical inputs
  → gate network
  → output truth table
  → half-adder glimpse
  → ALU path
```

Include one series-contact AND and one parallel-contact OR.

Observe

- complete at least one prediction set in the HTML lab;
- run either the Python or C truth-table program;
- record one prediction that was wrong and why.

Build

Using only NAND, draw:

1. NOT;

2. AND;
3. OR.

Then derive the truth table of:

$$Y = (A \text{ XOR } B) \text{ AND } (\text{NOT } C)$$

Do not consult a generated answer until every one of the eight input rows has an attempted output.

Mental model at the end of Week 3

```
continuous electrical behavior
    ↓ threshold convention
logical input bits
    ↓ transistor network topology
gate behavior
    ↓ composition
Boolean expression / combinational circuit
    ↓ repeated one-bit structures
arithmetic, comparison, selection
    ↓ state + control added later
ALU and CPU
```

The essential sentence is:

A Boolean function is not a thought occurring inside the circuit. It is our compact description of how an arranged physical network maps every allowed input state to an output state.

Why? notebook

1. Why is one isolated switch a state-holder or path-controller, but not yet a general computer?
 2. Where exactly does AND “exist”: in the expression, truth table, circuit topology, voltage behavior, or all of these at different layers?
 3. Why must a gate restore its output toward valid voltage ranges?
 4. Why can two structurally different circuits implement the same function?
 5. Why does functional completeness not imply that every real chip uses only one physical gate type?
 6. Why can a half-adder not yet add arbitrary multi-bit numbers?
 7. What new capability is needed before a circuit can remember a previous result?
-

References used selectively

Local reading:

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 10–12, from “Logic and Switches” through the opening of “A Binary Adding Machine” (local PDF pages 98–145). Local catalog entry: [source-materials/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf](#)
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, Chapter 3, §§3.1–3.3 and §3.3.5, on transistor switches, gates, De Morgan's law, combinational logic, and logical completeness (book pages 51–64). Local catalog entry: [source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf](#)

Historical and primary context:

- University of St Andrews, MacTutor biography of George Boole and the development of algebraic logic: <https://mathshistory.st-andrews.ac.uk/Biographies/Boole/>
- MIT Libraries, Claude Shannon's thesis, *A Symbolic Analysis of Relay and Switching Circuits*: <https://dspace.mit.edu/handle/1721.1/11173>
- Computer History Museum, the Bell Labs search for a semiconductor replacement for relays and vacuum tubes: <https://www.computerhistory.org/siliconengine/invention-of-the-point-contact-transistor/>
- NAND2Tetris, Boolean logic and gate-construction material: <https://www.nand2tetris.org/course>

Next bridge: combine gates into wider arithmetic and selection circuits, then ask what physical mechanism can preserve a result after the inputs change.